Speech Recognition using Mel-Frequency Cepstral Coefficients (MFCCs)

Lab 12

Department of Artificial Intelligence
NUML — Islamabad

Submitted by: **Junaid Asif**
Roll No: **BSAI–144**
Semester: **7th**

December 16, 2025

1 Introduction

Speech recognition enables machines to identify and interpret spoken language. A critical component of speech recognition systems is the extraction of meaningful features from raw audio signals. Mel-Frequency Cepstral Coefficients (MFCCs) are widely used because they effectively model the human auditory perception by focusing on perceptually important frequency components.

This lab introduces the complete pipeline for isolated word speech recognition using MFCC features and classical machine learning classifiers.

2 Objective

To implement a basic speech recognition system capable of identifying isolated spoken words using MFCC features and simple classifiers such as K-Nearest Neighbors (KNN) and Support Vector Machines (SVM).

3 Prerequisites

- Fundamentals of digital signal processing
- Understanding of MFCC feature extraction
- Knowledge of machine learning classifiers
- MATLAB with Audio Toolbox and Statistics Toolbox

4 Lab Task 1: MFCC Feature Preparation and Visualization

4.1 Description

This task focuses on extracting MFCC features, visualizing them, applying feature normalization, and aggregating them into a single vector suitable for classification.

4.2 MATLAB Code (Lab Task 1)

```
% lab_task_1.m
clear; clc; close all;

audioFolderPath = 'audio/';
sampleAudioFile = 'yes_1.wav';
n_mfcc_coeffs = 13;

[y, fs] = audioread(fullfile(audioFolderPath, sampleAudioFile));
y = mean(y,2);
y = y / max(abs(y));

afe = audioFeatureExtractor( ...
    'SampleRate', fs, ...
```

```
    'Window', hann(round(0.025*fs),'periodic'), ...
    'OverlapLength', round(0.010*fs), ...
    'mfcc', true, ...
    'NumBands', 26, ...
    'NumCoefficients', n_mfcc_coeffs);

features = extract(afe, y);

figure;
subplot(2,1,1);
plot((0:length(y)-1)/fs, y);
title('Speech Waveform');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(2,1,2);
imagesc(features');
axis xy; colorbar;
title('MFCC Features');
xlabel('Frame Index'); ylabel('MFCC Coefficient');

mean_features = mean(features,1);
std_features = std(features,0,1);
normalized_features = (features-mean_features)./(std_features+eps);

figure;
imagesc(normalized_features');
axis xy; colorbar;
title('Normalized MFCC Features');

aggregated_feature = mean(normalized_features,1);
disp('Aggregated Feature Vector:');
disp(aggregated_feature);
```

4.3 Explanation

MFCCs are extracted using a 25 ms window and 10 ms overlap. Feature normalization ensures each coefficient has zero mean and unit variance, improving classifier performance. Aggregation simplifies the time-varying MFCCs into a fixed-length feature vector.

5 Lab Task 2: Building a Simple Classifier

5.1 Description

In this task, aggregated MFCC features are used to train and evaluate a KNN and SVM classifier for isolated word recognition.

5.2 MATLAB Code (Lab Task 2)

```
% lab_task_2.m
clear; clc; close all;

audioFolderPath = 'audio/';
wordList = {'yes','no','up','down'};
```

```
n_mfcc_coeffs = 13;

allFeatures = [];
labels = [];
labelMap = containers.Map;
labelID = 0;

for i = 1:length(wordList)
    word = wordList{i};
    labelID = labelID + 1;
    labelMap(word) = labelID;

    files = dir(fullfile(audioFolderPath,[word '*.wav']));
    for j = 1:length(files)
        [y, fs] = audioread(fullfile(audioFolderPath,files(j).name));
        y = mean(y,2);
        y = y / max(abs(y));

        afe = audioFeatureExtractor( ...
            'SampleRate', fs, ...
            'Window', hann(round(0.025*fs),'periodic'), ...
            'OverlapLength', round(0.010*fs), ...
            'mfcc', true, ...
            'NumCoefficients', n_mfcc_coeffs);

        features = extract(afe, y);
        features = (features-mean(features,1))./(std(features,0,1)+eps);
        aggFeature = mean(features,1);

        allFeatures = [allFeatures; aggFeature];
        labels = [labels; labelID];
    end
end

cv = cvpartition(labels,'HoldOut',0.3);
XTrain = allFeatures(training(cv),:);
yTrain = labels(training(cv));
XTest = allFeatures(test(cv),:);
yTest = labels(test(cv));

knnModel = fitcknn(XTrain,yTrain,'NumNeighbors',3);
yPred = predict(knnModel,XTest);
accuracy_knn = mean(yPred==yTest);

svmModel = fitcsvm(XTrain,yTrain,'KernelFunction','linear');
yPredSVM = predict(svmModel,XTest);
accuracy_svm = mean(yPredSVM==yTest);

fprintf('KNN Accuracy: %.2f%%\n', accuracy_knn*100);
fprintf('SVM Accuracy: %.2f%%\n', accuracy_svm*100);
```

6 Questions and Discussion

6.1 KNN Advantages and Disadvantages

KNN is simple and effective for small datasets but becomes computationally expensive as dataset size grows. The value of k controls the smoothness of the decision boundary.

6.2 KNN vs SVM

KNN is instance-based and non-parametric, whereas SVM constructs an optimal decision boundary. SVMs generalize better for higher-dimensional data.

6.3 Importance of Train-Test Split

The train-test split ensures unbiased evaluation of the classifier's performance on unseen data, preventing overfitting.

7 Conclusion

This lab demonstrated a complete MFCC-based speech recognition pipeline using classical machine learning techniques. The system successfully classified isolated spoken words, highlighting the effectiveness of MFCC features in speech recognition applications.